

Pricing UI — Complete Report

Part A — UI Implementation

I created the same pricing table UI in two ways: Plain CSS and Tailwind CSS. Both versions contain three pricing plans (Basic, Pro, and Business), a highlighted Pro plan, responsive layout, hover effects, dark mode, pricing buttons, and features.

Measurement	Plain CSS	Tailwind CSS
Lines of CSS written by hand	300+	200+ HTML
Final CSS bundle size after build/minification	5 KB	HTML - 8 KB
Build time	40 minutes	35 minutes
Purple colour + middle-tier badge change	4 minutes	4 minutes

The Plain CSS version required more handwritten CSS because I had to write the layout, spacing, colours, responsive styles, hover effects, and dark mode myself. Tailwind reduced the amount of custom CSS because most of the styling was done using utility classes.

Part B — Dynamic Class Bug

I created a component where the background colour depends on a variable.

```
const color = "blue";
const className = `bg-${color}-500`;
```

This creates: **bg-blue-500**.

The class is created dynamically. Tailwind scans the source files during the production build and looks for complete class names. It can detect **bg-blue-500** when it is written directly, but it may not detect ``bg-${color}-500`` because the complete class name is not written in the source. As a result, the required CSS may not be included in the production build.

Development: The blue background can work. **Production:** The background can disappear because the required class was not generated.

Fix 1 — Use complete class names

```
const colors = {
  blue: "bg-blue-500",
  red: "bg-red-500",
  green: "bg-green-500"
};
```

I then use `colors[color]`. Tailwind can detect all the complete class names and include them in the production CSS.

Fix 2 — Safelist the classes

```
bg-blue-500
bg-red-500
bg-green-500
```

Another solution is to explicitly tell Tailwind which dynamic classes should be included. This prevents the required classes from being removed from the production build.

Part C — Refactor for Reuse

The pricing card was repeated three times for Basic, Pro, and Business. Since the structure is almost the same, I can create one reusable React component.

```
function PricingCard({ name, price }) {
  return (
    <div className="card">
      <h2>{name}</h2>
      <p>${price}/month</p>
      <button>Choose Plan</button>
    </div>
  );
}

<PricingCard name="Basic" price="9" />
<PricingCard name="Pro" price="19" />
<PricingCard name="Business" price="39" />
```

The output is still three pricing cards, but I only need to write the card structure once.

Using @apply

```
.card {
  @apply flex flex-col rounded-xl bg-white p-8;
}

<div class="card">
  <h2>Basic</h2>
  <p>$9/month</p>
  <button>Choose Plan</button>
</div>
```

The same styling can be used for the other cards.

Comparison — 200 Words

The reusable component and @apply both help reduce repetition, but they solve different problems. A reusable component creates one common structure for the pricing card. The name, price, features, and other information can be passed to the component as values. This means I can create many pricing cards without copying the complete HTML each time. If I need to change the card structure later, I only need to change it in one place.

@apply is mainly useful for grouping repeated Tailwind styles. For example, I can create a .card class containing common Tailwind utilities such as padding, border radius, background colour, and flexbox properties. This makes the styling easier to reuse, but the HTML structure still needs to be repeated.

For a small static page, @apply can be simple and useful. However, for a real application, I would choose the reusable component approach. Components are easier to reuse when the application becomes larger and when cards need different data or behaviour. I would still use Tailwind classes for styling and use @apply when a group of styles is repeated often. For a product that will be developed for two years, reusable components would be my main choice because they make future changes easier and reduce repeated structure.

Part D — Configuration

I extended the Tailwind theme instead of replacing it. I added a brand colour palette, a custom font, and one extra spacing value.

```
theme: {
  extend: {
    colors: {
      brand: {
        primary: "#3e556e",
        secondary: "#6499a2"
      }
    },
    fontFamily: {
```

```

        brand: ["Poppins", "sans-serif"]
    },

    spacing: {
        18: "4.5rem"
    }
}

```

Brand colours: I added brand-primary and brand-secondary for the main colours of the website.

Custom font: I added font-brand, which uses the Poppins font.

Extra spacing: I added 18 = 4.5rem as an additional spacing value.

Extend vs replacing: I used extend because I wanted to add my custom values to Tailwind's existing theme. With extend, the default Tailwind values and my custom values are both available. If I replace the theme instead, I can remove or overwrite Tailwind's default values. Therefore, extend is better when I want to add new values without losing the existing options.

Part E — Tailwind Discussion

A senior developer might say that **Tailwind is just inline CSS with extra steps**. Their main point is that we write many classes directly inside the HTML. For example, a button might have classes such as bg-blue-500, p-3, rounded-lg, and text-white. This can make the HTML long and difficult to read. With normal CSS, we can give an element a simple class such as .button and keep the styling in a separate CSS file.

I understand this argument, but I do not completely agree. Tailwind is different from inline styles because its classes follow a fixed and consistent design system. It provides common spacing, colours, font sizes, responsive styles, hover effects, and dark mode. Developers do not need to create a new CSS rule for every small change. Tailwind also helps keep the design consistent across the whole project.

For a **5-person team building a product for 2 years**, I would choose Tailwind. A small team can work faster because everyone can use the same classes and design rules. It can also reduce the amount of custom CSS that the team needs to maintain.

However, I would not use Tailwind for everything. For repeated or complicated parts of the website, I would create reusable components. I would also use normal CSS when something is difficult or messy to create with Tailwind.

My final decision is to use Tailwind with reusable components. I think this gives a good balance between fast development, consistent design, and easy maintenance for a long-term project.